

OpenSeek Closest Integers 技术报告

1. 任务概述

本任务对应 OpenSeek 系列中的 **Closest Integers** 问题，核心目标是：给定一个整数列表，计算列表中任意两个整数之间的最小绝对差，并将结果以字符串形式输出。

输入数据通常表现为字符串格式的整数列表，例如：

```
1 "[59, 26, -96, -30]"
```

程序需要将该字符串解析为整数列表，然后在所有整数对中寻找绝对差最小的一组。对于上述示例，排序后列表为：

```
1 [-96, -30, 26, 59]
```

相邻元素差值分别为：

```
1 66, 56, 33
```

因此最小绝对差为 33，最终输出：

```
1 "33"
```

本任务虽然表面上可以通过两层循环暴力枚举所有整数对完成，但考虑到测试集规模、运行效率以及提交稳定性，代码采用了排序后比较相邻元素的方式完成计算。该方法利用了一个基本性质：对于一组数字，任意两个数之间的最小绝对差一定出现在排序后的相邻元素之间。因此，只需要对列表排序，然后线性扫描相邻差值即可。

2. 数据集结构与输入输出格式

数据文件为 JSON 格式，代码通过 `task1_data_loader` 函数读取数据。数据文件中主要包含以下字段：

```
1 task_id
2 task_name
3 Definition
4 examples
5 test_samples
```

其中：

`task_id` 表示当前任务编号，用于标识任务来源。

`task_name` 表示任务名称，本任务为 Closest Integers。

`Definition` 保存任务自然语言描述，主要说明输入、输出以及任务目标。

`examples` 表示训练样例或参考样例，每条样例一般包含输入和标准输出。代码会使用这些样例对模型生成的 `solve` 函数进行预验证。

`test_samples` 表示最终需要预测的测试样例。程序不会读取测试样例的标准答案，而是调用生成或回退的求解器计算预测结果。

每条测试样例的输入字段为字符串形式的整数列表。最终程序输出 JSONL 文件，每一行对应一个测试样例，格式如下：

```
1 {"test_sample_id": "xxx", "prediction": "33"}
```

其中 `test_sample_id` 来自原测试样例的 `id` 字段，`prediction` 是程序计算得到的最小绝对差结果，统一保存为字符串。

3. 总体设计思路

本代码不是直接手写单一求解函数后处理测试集，而是采用了“模型生成求解器 + 自动清洗 + 安全验证 + 样例验证 + 回退求解器 + 批量预测”的整体流程。

整体流程如下：

```
1  读取 JSON 数据
2      ↓
3  构造代码生成提示词
4      ↓
5  调用 Qwen3-4B 生成 solve 函数
6      ↓
7  提取并清洗生成代码
8      ↓
9  安全检查与受限环境编译
10     ↓
11 使用 examples 前 200 条验证准确率
12     ↓
13 若生成函数可用，则用于测试集预测
14 若生成函数不可用，则回退到内置 fallback_solve
15     ↓
16 多线程处理 test_samples
17     ↓
18 生成 JSONL 提交文件
```

该设计兼顾了两点：

第一，符合任务希望利用大模型生成代码的思路。程序会根据任务定义和若干样例，自动构造 prompt，并请求 Qwen3-4B 生成一个符合接口要求的 `solve(input_text: str) -> str` 函数。

第二，保证提交结果的稳定性。由于大模型生成代码可能出现格式错误、解释性文字、非法 import、函数名不符合要求、运行异常等问题，程序额外设计了代码清洗、安全检查、编译验证和 fallback 回退机制。这样即使模型生成失败，也可以使用内置确定性算法继续完成测试集预测。

4. 核心算法原理

本任务要求计算整数列表中任意两个整数的最小绝对差。直接做法是枚举所有二元组：

```
1 for i in range(n):
2     for j in range(i + 1, n):
3         ans = min(ans, abs(nums[i] - nums[j]))
```

该方法时间复杂度为 $O(n^2)$ ，当列表较长时效率较低。

本代码采用排序优化方法：

1. 将输入字符串解析为整数列表；
2. 将整数列表按升序排序；
3. 扫描排序后相邻元素；
4. 计算相邻元素差值；
5. 取所有相邻差值中的最小值；
6. 返回字符串结果。

其原因在于：排序后，如果两个元素之间不是相邻关系，那么它们中间至少存在一个其他元素。中间元素与两端某个元素之间的距离一定不会大于两端元素之间的距离。因此，最小绝对差只可能出现在排序后的相邻元素之间。

例如：

```
1 原列表: [10, -2, 3, 8]
2 排序后: [-2, 3, 8, 10]
3 相邻差值: 5, 5, 2
4 最小绝对差: 2
```

因此，算法复杂度为：

```
1 排序:  $O(n \log n)$ 
2 扫描:  $O(n)$ 
3 总复杂度:  $O(n \log n)$ 
```

空间复杂度主要取决于排序过程和列表存储，通常为 $O(n)$ 或 $O(1)$ 到 $O(n)$ ，具体取决于 Python 排序实现和输入转换过程。

5. 代码模块设计

5.1 数据加载模块

代码中的 `task1_data_loader(file_path)` 负责读取任务 JSON 文件。该函数使用 `json.load` 解析输入文件，并返回任务 ID、任务名称、任务定义、训练样例和测试样例。

其主要作用是将原始 JSON 数据转换为后续流程可直接使用的 Python 对象。返回内容包括：

```
1 task_id
2 task_name
3 definition_list
4 examples
5 test_samples
```

该模块是整个流程的入口。后续 prompt 构造、样例验证和测试集预测都依赖它提供的数据。

5.2 模型调用模块

代码使用 OpenAI SDK 初始化了一个兼容 OpenAI 接口格式的客户端，然后通过 `qwen_api` 函数调用 Qwen3-4B 模型。

该函数的输入是 `messages`，即聊天模型所需的消息列表。函数内部设置了：

```
1 model="/Qwen3-4B/Qwen/Qwen3-4B"
2 temperature=0.0
3 retries=3
```

其中，`temperature=0.0` 表示模型输出尽可能确定，减少随机性。由于本任务目标是生成可执行代码，而不是开放式文本创作，因此低温度设置更适合保证输出稳定。

`retries=3` 表示如果模型调用失败，程序最多尝试三次。每次失败后会等待 2 秒再重试。如果最终仍失败，则返回空字符串，并在控制台打印 API 调用失败信息。

该模块解决的是模型输出来源问题，为后续自动生成 `solve` 函数提供基础。

5.3 代码生成提示词设计

代码中包含两个层次的 prompt。

第一部分是系统提示词 `CODEGEN_SYSTEM_PROMPT`。

该提示词明确要求模型扮演严格的 Python 算法代码生成器，并规定：

```
1 只输出可执行 Python 代码
2 不要解释
3 不要 markdown
4 不要代码块标记
5 只定义一个 solve(input_text: str) -> str 函数
6 不要 import
7 不要 main
8 不要示例
9 不要调用 open、eval、exec、compile、__import__
10 输入是字符串格式的整数列表
11 输出必须是字符串
```

这部分约束非常重要。因为模型默认可能会输出解释文字、Markdown 代码块，或者添加 `import`、测试样例、主函数等内容。这些内容都会影响后续自动编译和调用。因此，系统提示词通过强约束将输出范围限制为一个单独的 `solve` 函数。

第二部分是用户提示词，由 `build_codegen_user_prompt` 函数动态构造。

该函数会把任务 ID、任务名称、任务定义和最多 8 条样例拼接成 prompt。例如：

```
1 任务ID: ...
2 任务名: ...
3
4 任务定义:
5 - ...
6
7 下面是若干样例:
8 输入: ...
9 输出: ...
10
11 请直接输出 solve(input_text: str) -> str 的完整 Python 代码。
12 再次强调:
13 - 不要 import
14 - 不要解释
15 - 只输出代码
```

这种设计的目的在于给模型提供足够的任务上下文，同时避免 prompt 过长。由于本任务逻辑明确，使用少量样例即可帮助模型理解输入输出格式。

5.4 生成代码提取模块

模型输出不一定完全符合要求。即使 prompt 中要求“不输出 markdown”，模型仍可能返回如下格式：

```
1  ```python
2  def solve(input_text: str) -> str:
3      ...
4  ```
```

因此，代码设计了 `extract_code(text)` 函数，用于从模型输出中提取真正的 Python 代码。

该函数首先检查文本中是否存在 Markdown 代码块。如果存在，则通过正则表达式提取代码块内部内容。如果不存在，则直接返回原文本。

这一步的作用是提高模型输出的容错性，使程序能够处理模型偶尔添加代码块标记的情况。

5.5 生成代码后处理模块

仅提取代码还不够，因为模型输出中可能仍然包含多余内容。例如：

```
1 下面是代码：
2  def solve(input_text: str) -> str:
3      ...
```

或者：

```
1  import ast
2
3  def solve(input_text: str) -> str:
4      ...
```

因此，代码进一步使用 `postprocess_generated_code(code)` 对生成结果进行清洗。该函数主要完成三类处理：

第一，删除 Markdown 残留标记，例如：

```
1  ```python
2  ```
```

第二，删除所有 `import` 和 `from ... import ...` 行。因为系统要求生成代码不要 `import`，而且后续执行环境已经预先提供了 `ast` 对象。如果允许模型自行 `import`，会增加不确定性和安全风险。

第三，如果模型在函数前输出了解释文字，则通过正则表达式定位：

```
1  def solve(input_text: str) -> str:
```

并只保留从该函数定义开始的内容。

通过这一步处理，程序可以将大模型输出转换为更规范、更容易编译的 Python 函数代码。

5.6 安全检查模块

由于模型生成的代码会被程序动态执行，因此必须进行安全检查。代码中的 `is_code_safe(code)` 函数用于检测生成代码中是否包含高风险内容。

禁止内容包括：

```
1  open(
2  eval(
3  exec(
4  compile(
5  __import__
6  os
7  sys
8  subprocess
9  pathlib
10 shutil
11 pickle
```

这些内容可能带来文件访问、系统调用、动态执行、模块导入或反序列化风险。虽然本任务只需要解析整数列表并计算最小差值，但模型生成代码存在不可控性，因此安全检查是必要环节。

如果检测到禁止模式，函数会返回：

```
1 | False, "生成代码包含不允许内容：..."
```

随后 `compile_solver` 会抛出异常，本次生成结果会被判定为不可用。

需要注意的是，这种检查属于基础安全过滤，主要用于本任务场景下的风险控制。它并不是完整的 Python 沙箱，但已经能拦截与本任务明显无关的高风险操作。

5.7 动态编译与受限执行环境

通过安全检查后，代码会进入 `compile_solver(code)` 函数进行动态编译。

该函数首先构造了一个受限的 `safe_builtins` 字典，只允许生成代码访问少量基础函数，例如：

```
1 | len
2 | min
3 | max
4 | sum
5 | abs
6 | str
7 | int
8 | float
9 | list
10 | dict
11 | set
12 | tuple
13 | sorted
14 | range
15 | enumerate
16 | zip
17 | map
18 | filter
19 | any
20 | all
```

同时，执行环境中提供了 `ast` 对象，允许生成代码使用：

```
1 | ast.literal_eval
```

解析输入字符串。

然后程序使用：

```
1 | exec(code, exec_globals, exec_locals)
```

执行生成代码，并从局部或全局命名空间中提取 `solve` 函数。

如果生成代码中没有定义 `solve` 函数，程序会抛出：

```
1 | 生成代码中未找到 solve 函数
```

这一模块的作用是将模型生成的文本代码转换为可调用的 Python 函数对象。后续无论是样例验证还是测试集预测，都会通过该函数对象完成。

5.8 内置回退求解器

考虑到模型生成代码可能失败，程序额外定义了 `fallback_solve(input_text: str) -> str` 作为保底方案。

其逻辑如下：

```
1 | nums = ast.literal_eval(input_text.strip())
2 | nums = sorted(int(x) for x in nums)
3 |
4 | if len(nums) < 2:
5 |     return "0"
6 |
7 | best = min(nums[i] - nums[i - 1] for i in range(1, len(nums)))
8 | return str(best)
```

该函数不依赖模型生成结果，完全按照本任务的确定性算法实现。它先使用 `ast.literal_eval` 安全解析输入字符串，再将列表元素转换为整数并排序。若列表长度小于 2，则返回 `"0"` 作为边界处理。否则，扫描排序后相邻元素，计算最小差值并返回字符串。

回退机制的意义在于保证程序稳定性。即使模型输出代码为空、格式错误、编译失败、验证不通过，程序仍然能够完成测试集预测，而不会因为生成式模型的不稳定性导致整个流程中断。

5.9 样例验证模块

程序使用 `validate_solver(solve_func, examples, limit=200)` 对生成的 `solve` 函数进行验证。

该函数最多取前 200 条 examples，逐条调用生成的 `solve_func`，并将预测结果与标准答案比较。

验证过程中，如果某条样例运行出错，程序会将预测值记录为：

```
1 | [ERROR] 错误信息
```

如果预测值与标准答案不一致，则将该样例加入 `error_cases`，保存内容包括：

```
1 | 样例 ID
2 | 输入
3 | 标准答案
4 | 预测答案
```


最终函数返回：

```
1 | acc, error_cases
```

其中 `acc` 是验证准确率，`error_cases` 是错误样例列表。

这一环节是代码可靠性的关键。因为大模型生成的代码即使能成功编译，也不一定逻辑正确。通过训练样例验证，可以在正式处理测试集之前筛掉错误求解器。

5.10 多次生成与最优选择机制

程序中的 `generate_valid_solver` 函数负责完整的求解器生成流程。

该函数最多尝试生成 `max_try=3` 次。每次流程为：

```
1 | 构造 messages
2 | 调用 qwen_api
3 | 提取代码
4 | 清洗代码
5 | 编译 solve 函数
6 | 使用 examples 验证
7 | 记录准确率
```

如果某一次生成代码编译失败，程序会打印：

```
1 | 第 i 次代码编译失败：...
```

然后继续下一次尝试。

如果某一次生成代码验证准确率达到 100%，程序会立即返回该版本代码和函数对象。

如果三次都没有达到 100%，但至少有一个版本能成功编译和验证，则返回准确率最高的版本。

如果所有模型生成结果都失败，则返回内置 `fallback_solve`，并记录：

```
1 | 模型生成代码均未通过验证，已回退
```

这一机制体现了“生成式方法 + 验证筛选”的思想。程序不是盲目信任模型第一次生成的代码，而是通过多次尝试和样例验证选择最可靠的函数。

6. 测试集预测流程

当求解器确定后，程序进入测试集预测阶段。

首先，主程序读取：

```
1 | eval_data = test_samples
```

然后使用 `functools.partial` 固定 `solve_func` 和 `task_id` 参数，得到可直接处理单条样例的函数：

```
1 process_func = partial(process_single_sample, solve_func=solve_func, task_id=task_id)
```

单条样例处理函数 `process_single_sample` 的逻辑为：

1. 读取样例 ID；
2. 读取输入字符串；
3. 调用 `solve_func(input_text)` 计算预测值；
4. 如果运行异常，则输出错误信息，并将预测值设为 `"0"`；
5. 返回包含任务 ID、样例 ID 和预测值的字典。

其返回结构为：

```
1 {  
2     "task_id": task_id,  
3     "sample_id": sample_id,  
4     "prediction": pred  
5 }
```

虽然最终提交文件中不会保留 `task_id` 字段，但该字段在内部结果列表中有助于保留任务上下文。

7. 多线程加速设计

代码使用 `ThreadPoolExecutor` 对测试集进行并行处理：

```
1 max_workers = 200
```

然后通过：

```
1 executor.map(process_func, eval_data)
```

并发处理所有测试样例。

本任务中每条样例之间完全独立，不存在共享状态修改，因此适合并行化处理。多线程可以减少大量测试样例逐条处理时的总耗时，尤其是在测试集规模较大时，能够明显提升输出效率。

同时，程序使用 `tqdm` 显示预测进度：

```
1 tqdm(..., total=total_cnt, desc="Predicting")
```

这样运行时可以直观看到处理进度，方便判断程序是否正常运行。

需要说明的是，本任务单条样例计算本身非常快，因此多线程主要起到批量调度和进度管理作用。在测试样例特别少的情况下，多线程收益可能不明显；但在大规模测试集场景中，该设计可以增强处理效率。

8. 输出文件生成

预测完成后，程序会将内部结果转换为最终提交格式：

```
1 output_data = [  
2     {  
3         "test_sample_id": item["sample_id"],  
4         "prediction": item["prediction"]  
5     }  
6     for item in results_list  
7 ]
```

最终每一行只保留两个字段：

```
1 test_sample_id  
2 prediction
```

然后程序使用：

```
1 os.makedirs(os.path.dirname(out_path), exist_ok=True)
```

确保输出目录存在，再逐行写入 JSONL 文件：

```
1 with open(out_path, 'w', encoding='utf-8') as f:  
2     for item in output_data:  
3         line = json.dumps(item, ensure_ascii=False)  
4         f.write(line + '\n')
```

使用 JSONL 格式的好处是每条预测结果独占一行，便于评测系统逐条读取，也便于后续调试和增量检查。

9. 异常处理与稳定性设计

本代码在多个关键位置加入了异常处理机制。

第一，API 调用阶段设置重试机制。模型调用失败不会立即终止程序，而是等待后重试。若最终失败，则返回空字符串。

第二，代码编译阶段捕获异常。如果模型生成代码不符合语法、安全检查失败或没有定义 `solve` 函数，程序会跳过该次生成，继续尝试下一次。

第三，样例验证阶段捕获运行异常。如果某条样例执行失败，程序会记录错误，而不是中断整体验证流程。

第四，测试集预测阶段也捕获异常。如果某个测试样例处理失败，程序会打印错误信息，并返回默认预测 `"0"`，保证最终 JSONL 文件仍可完整生成。

第五，整体求解器层面提供 fallback。若模型生成求解器不可用，程序会切换到内置确定性求解器，避免生成模型失败导致提交失败。

这些设计共同提高了程序在真实运行环境中的鲁棒性。

